



Hands-on Exercise 3: Programming Interactive Data Visualisation with R

AUTHOR

PHAM Hung Son

PUBLISHED

January 26, 2026

MODIFIED

January 26, 2026

1 Learning Outcome

In this hands-on exercise, you will learn how to create interactive data visualisation by using functions provided by **ggiraph** and **plotlyr** packages.

2 Getting started

2.1 Installing and loading the required libraries

First, we write code chunk to install and launch the following R packages:

- [ggiraph](#) for making ‘ggplot’ graphics interactive.
- [plotly](#), R library for plotting interactive statistical graphs.
- [DT](#) provides an R interface to the JavaScript library [DataTables](#) that create interactive table on html page.
- [tidyverse](#), a family of modern R packages specially designed to support data science, analysis and communication task including creating static statistical graphs.
- [patchwork](#) for combining multiple ggplot2 graphs into one figure.

The code chunk below will be used to accomplish the task.

```
pacman::p_load(ggiraph, plotly, patchwork, DT, tidyverse)
```

3 Importing data

In this section, *Exam_data.csv* provided will be used. Using [read_csv\(\)](#) of **readr** package, import *Exam_data.csv* into R.

The code chunk below `read_csv()` of **readr** package is used to import *Exam_data.csv* data file into R and save it as an tibble data frame called `exam_data`.

```
exam_data <- read_csv("data/Exam_data.csv")
```

4 Interactive Data Visualisation - ggiraph methods

[ggiraph](#)  is an htmlwidget and a ggplot2 extension. It allows ggplot graphics to be interactive.

Interactive is made with [ggplot geometries](#) that can understand three arguments:

- **Tooltip:** a column of data-sets that contain tooltips to be displayed when the mouse is over elements.
- **OnClick:** a column of data-sets that contain a JavaScript function to be executed when elements are clicked.
- **Data_id:** a column of data-sets that contain an id to be associated with elements.

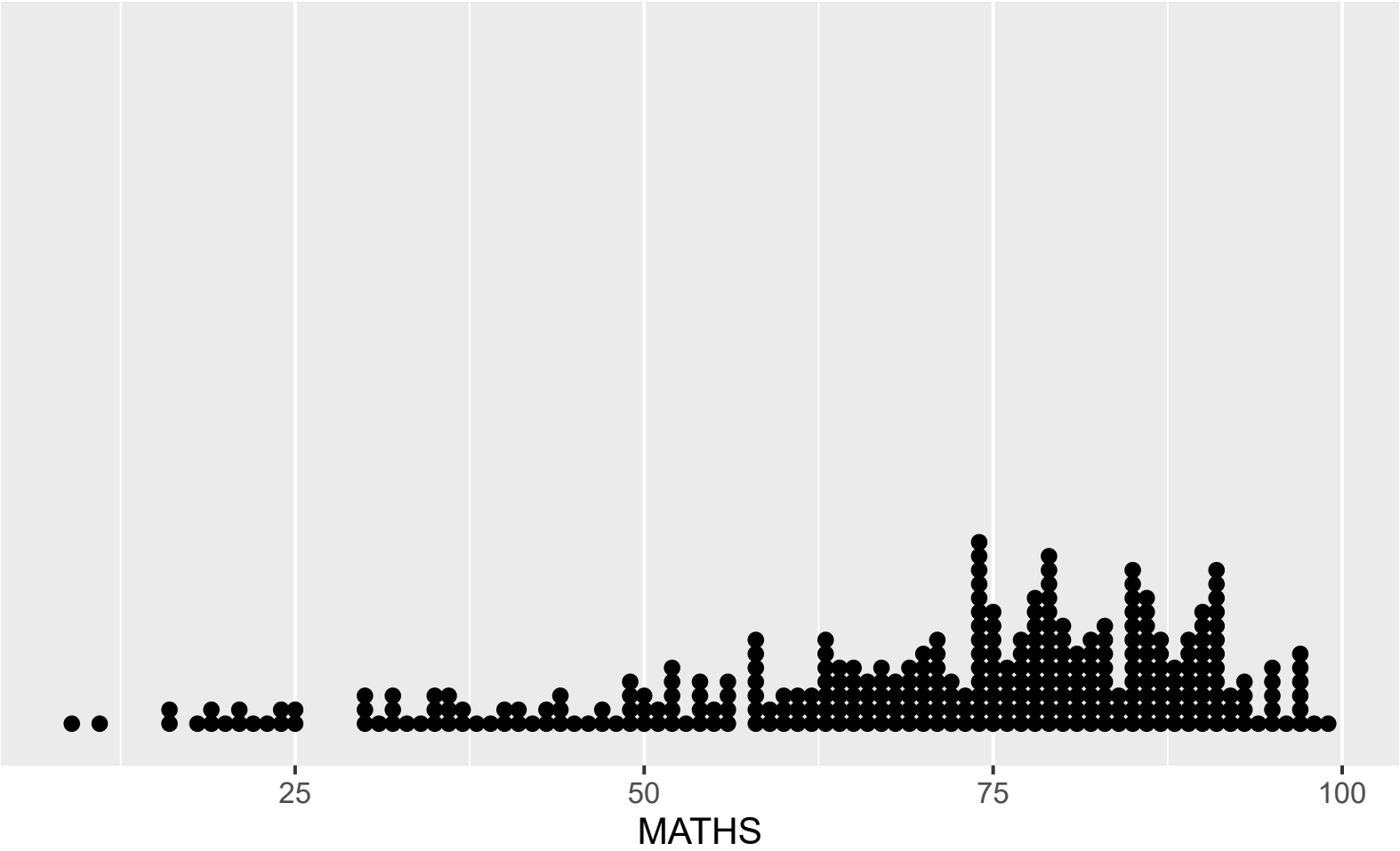
If it used within a shiny application, elements associated with an id (data_id) can be selected and manipulated on client and server sides. Refer to this [article](#) for more detail explanation.

4.1 Tooltip effect with *tooltip* aesthetic

Below shows a typical code chunk to plot an interactive statistical graph by using **ggiraph** package. Notice that the code chunk consists of two parts. First, an ggplot object will be created. Next, [girafe\(\)](#) of **ggiraph** will be used to create an interactive svg object.

```
p <- ggplot(data=exam_data,
  aes(x = MATHS)) +
  geom_dotplot_interactive(
    aes(tooltip = ID),
    stackgroups = TRUE,
    binwidth = 1,
    method = "histodot") +
  scale_y_continuous(NULL,
    breaks = NULL)

girafe(
  ggobj = p,
  width_svg = 6,
  height_svg = 6*0.618
)
```



Notice that two steps are involved. First, an interactive version of ggplot2 geom (i.e. [geom_dotplot_interactive\(\)](#)) will be used to create the basic graph. Then, [girafe\(\)](#) will be used to generate an svg object to be displayed on an html page.

5 Interactivity

By hovering the mouse pointer on an data point of interest, the student’s ID will be displayed.

5.1 Displaying multiple information on tooltip

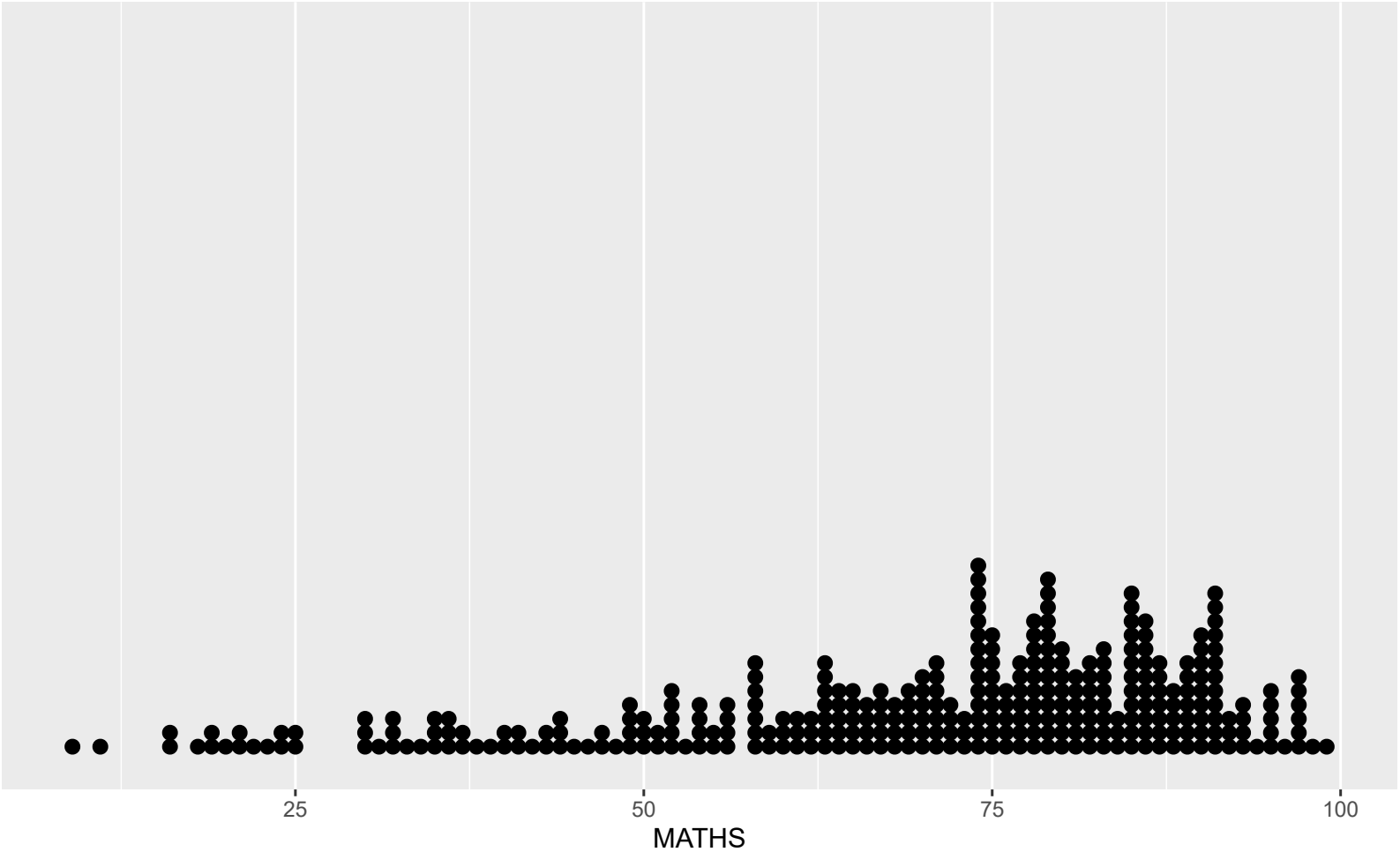
The content of the tooltip can be customised by including a list object as shown in the code chunk below.

```
exam_data$tooltip <- c(paste0(
  "Name = ", exam_data$ID,
  "\n Class = ", exam_data$CLASS))

p <- ggplot(data=exam_data,
  aes(x = MATHS)) +
  geom_dotplot_interactive(
    aes(tooltip = exam_data$tooltip),
    stackgroups = TRUE,
    binwidth = 1,
    method = "histodot") +
```

```
scale_y_continuous(NULL,
                    breaks = NULL)

girafe(
  ggobj = p,
  width_svg = 8,
  height_svg = 8*0.618
)
```



The first three lines of codes in the code chunk create a new field called *tooltip*. At the same time, it populates text in ID and CLASS fields into the newly created field. Next, this newly created field is used as tooltip field as shown in the code of line 7.

6 Interactivity

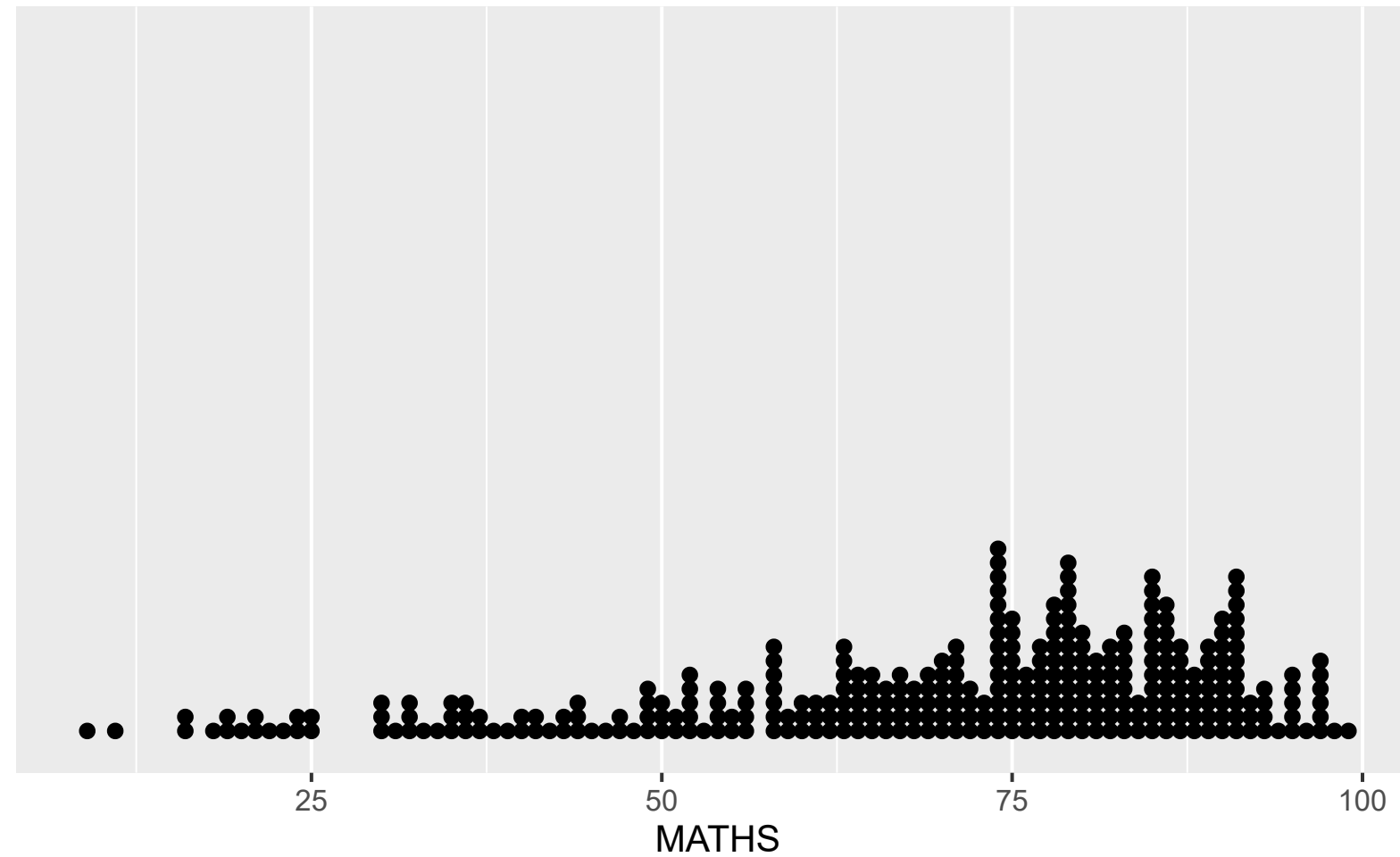
By hovering the mouse pointer on an data point of interest, the student’s ID and Class will be displayed.

6.1 Customising Tooltip style

Code chunk below uses `opts_tooltip()` of **ggiraph** to customize tooltip rendering by add css declarations.

```
tooltip_css <- "background-color:white; #<<
font-style:bold; color:black;" #<<
```

```
p <- ggplot(data=exam_data,
  aes(x = MATHS)) +
  geom_dotplot_interactive(
    aes(tooltip = ID),
    stackgroups = TRUE,
    binwidth = 1,
    method = "histodot") +
  scale_y_continuous(NULL,
    breaks = NULL)
girafe(
  ggobj = p,
  width_svg = 6,
  height_svg = 6*0.618,
  options = list( #<<
    opts_tooltip( #<<
      css = tooltip_css)) #<<
)
```



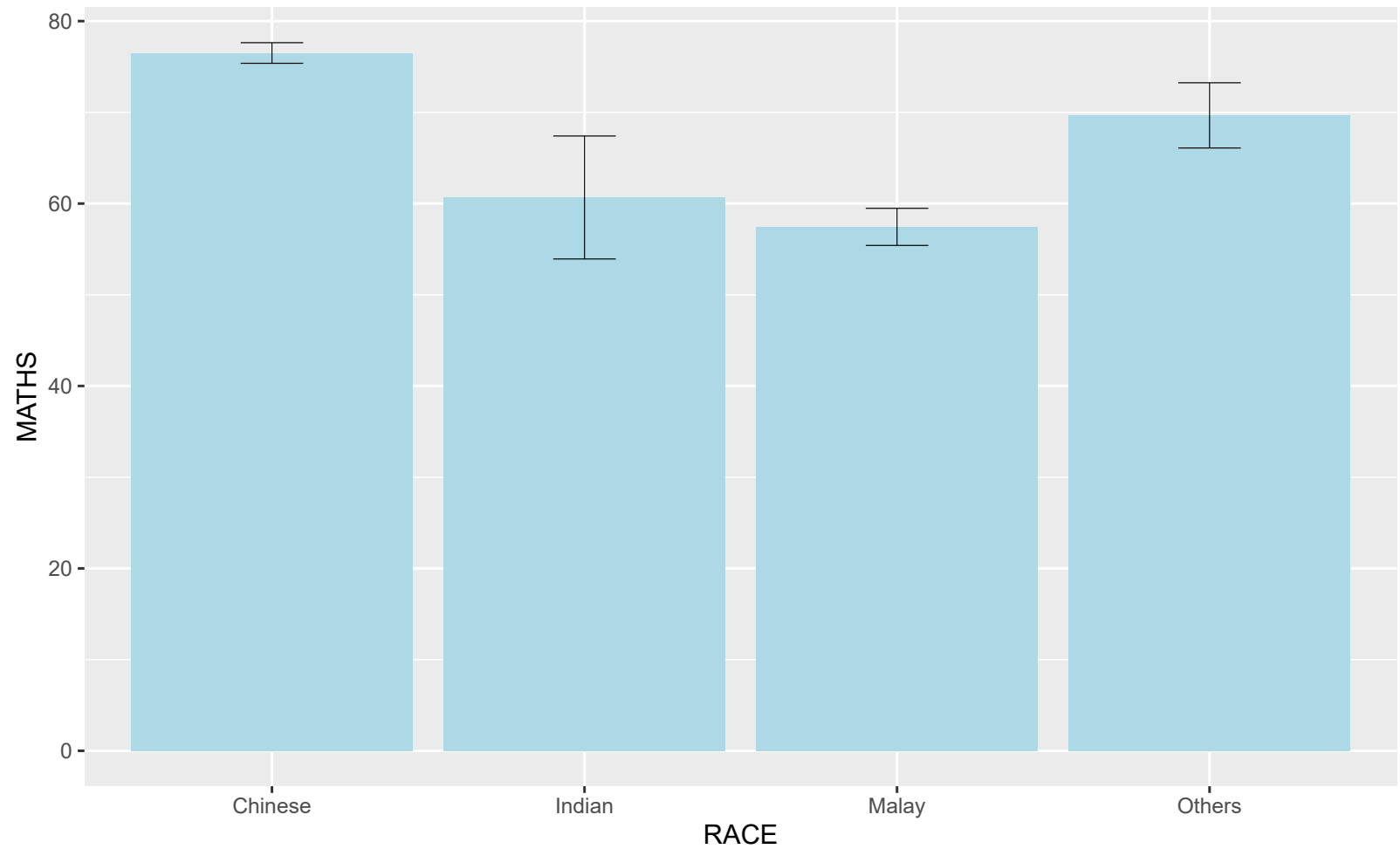
Notice that the background colour of the tooltip is black and the font colour is white and bold.

Refer to [Customizing girafe objects](#) to learn more about how to customise ggiraph objects.

6.2 Displaying statistics on tooltip

Code chunk below shows an advanced way to customize tooltip. In this example, a function is used to compute 90% confident interval of the mean. The derived statistics are then displayed in the tooltip.

```
tooltip <- function(y, ymax, accuracy = .01) {  
  mean <- scales::number(y, accuracy = accuracy)  
  sem <- scales::number(ymax - y, accuracy = accuracy)  
  paste("Mean maths scores:", mean, "+/-", sem)  
}  
  
gg_point <- ggplot(data=exam_data,  
                  aes(x = RACE),  
) +  
  stat_summary(aes(y = MATHS,  
                  tooltip = after_stat(  
                    tooltip(y, ymax))),  
              fun.data = "mean_se",  
              geom = GeomInteractiveCol,  
              fill = "light blue"  
) +  
  stat_summary(aes(y = MATHS),  
              fun.data = mean_se,  
              geom = "errorbar", width = 0.2, size = 0.2  
)  
  
girafe(ggobj = gg_point,  
       width_svg = 8,  
       height_svg = 8*0.618)
```

6.3 Hover effect with *data_id* aesthetic

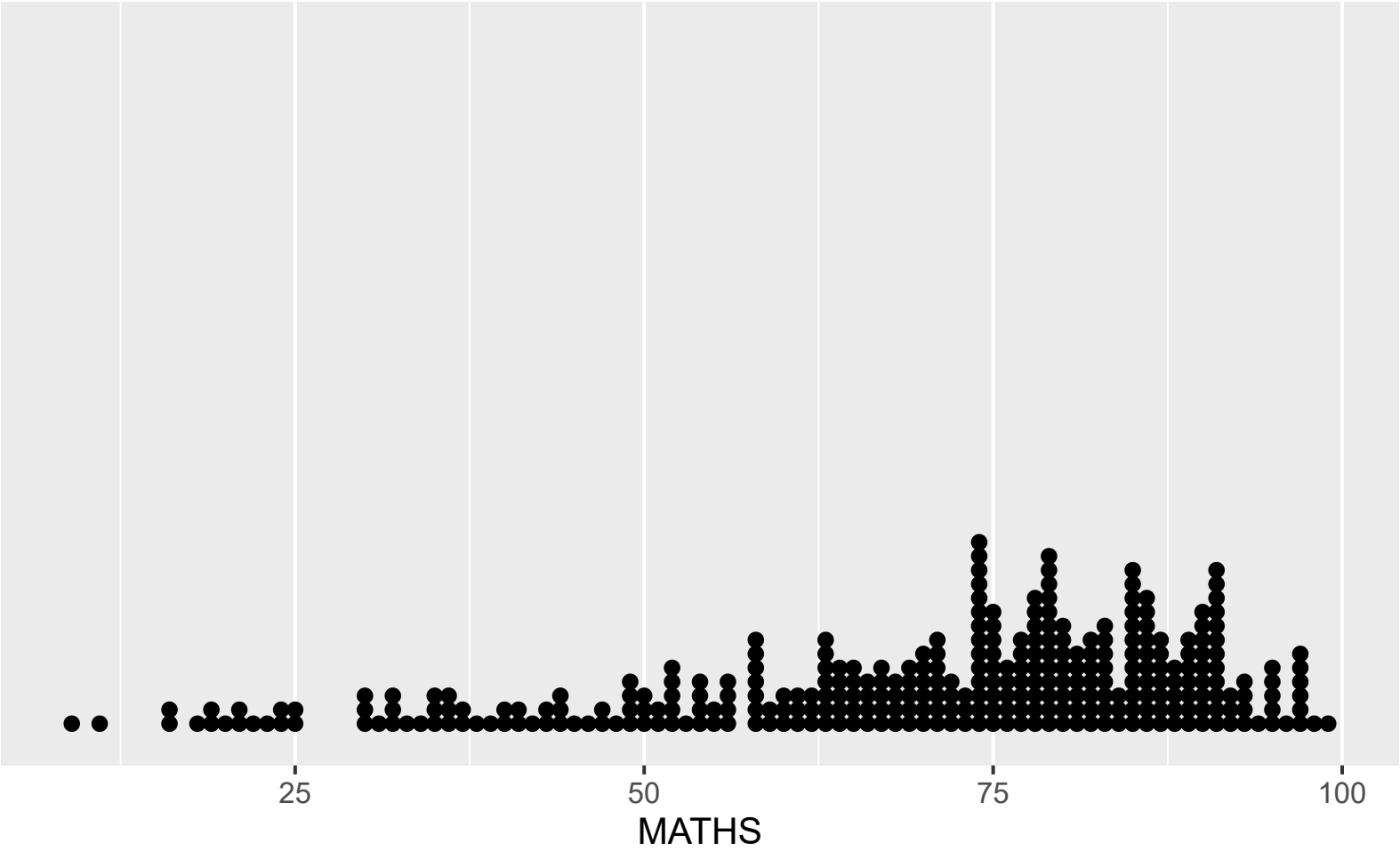
Code chunk below shows the second interactive feature of ggiraph, namely `data_id`.

Interactivity: Elements associated with a *data_id* (i.e CLASS) will be highlighted upon mouse over.

Note that the default value of the hover css is `hover_css = "fill:orange;"`.

```
p <- ggplot(data=exam_data,
  aes(x = MATHS)) +
  geom_dotplot_interactive(
    aes(data_id = CLASS),
    stackgroups = TRUE,
    binwidth = 1,
    method = "histodot") +
  scale_y_continuous(NULL,
    breaks = NULL)

girafe(
  ggobj = p,
  width_svg = 6,
  height_svg = 6*0.618
)
```



6.4 Styling hover effect

In the code chunk below, css codes are used to change the highlighting effect.

Interactivity: Elements associated with a *data_id* (i.e CLASS) will be highlighted upon mouse over.

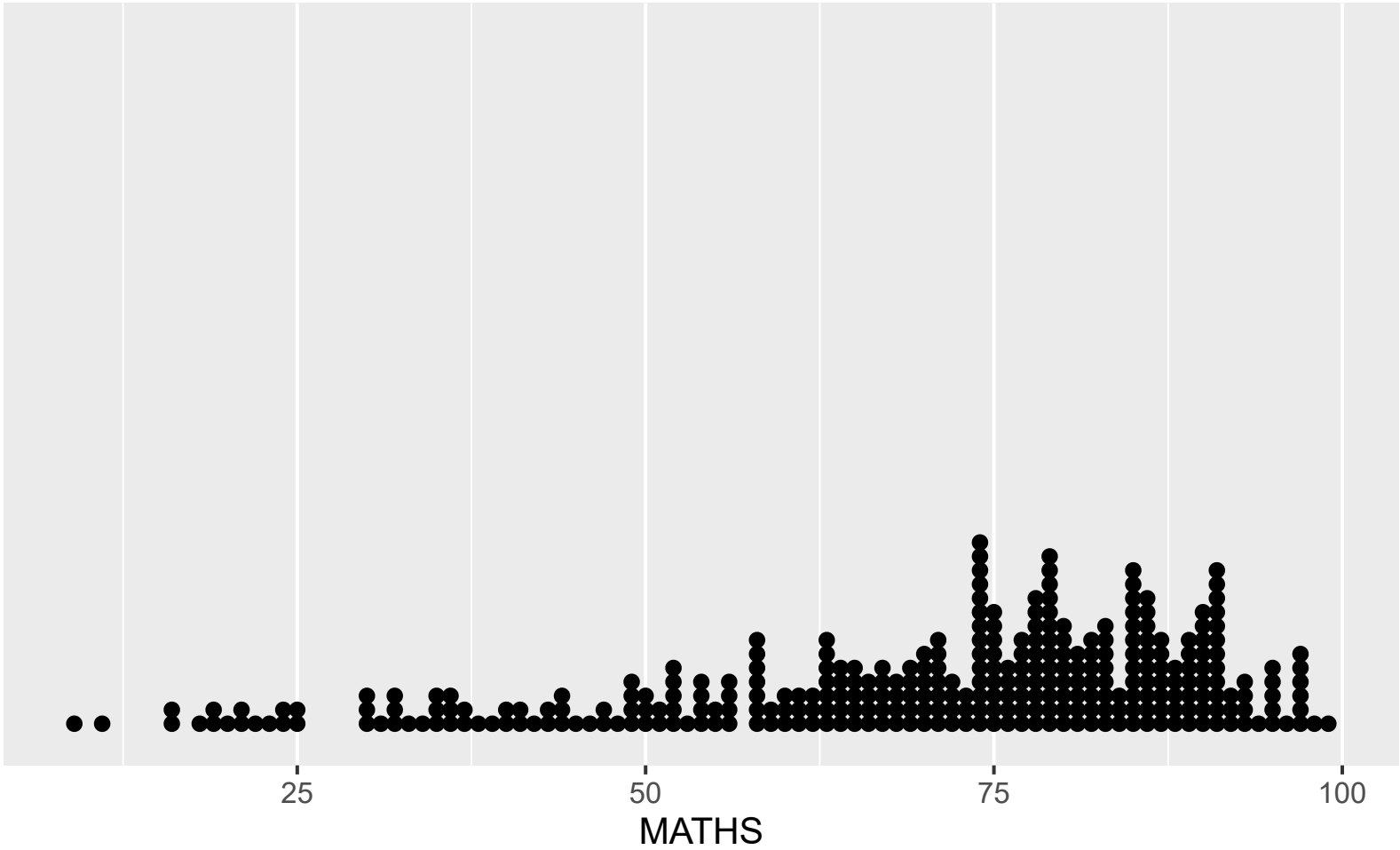
Note: Different from previous example, in this example the ccs customisation request are encoded directly.

```
p <- ggplot(data=exam_data,
  aes(x = MATHS)) +
  geom_dotplot_interactive(
    aes(data_id = CLASS),
    stackgroups = TRUE,
    binwidth = 1,
    method = "histodot") +
  scale_y_continuous(NULL,
    breaks = NULL)

girafe(
  ggobj = p,
  width_svg = 6,
  height_svg = 6*0.618,
  options = list(
```



```
opts_hover(css = "fill: #202020;"),
opts_hover_inv(css = "opacity:0.2;")
)
)
```



i **Note**

- `axis_title_size` argument is used to increase the font size of the axis title to 18,
- `base_size` argument is used to increase the default axis label to 15, and
- `grid` argument is used to remove the x-axis grid lines.

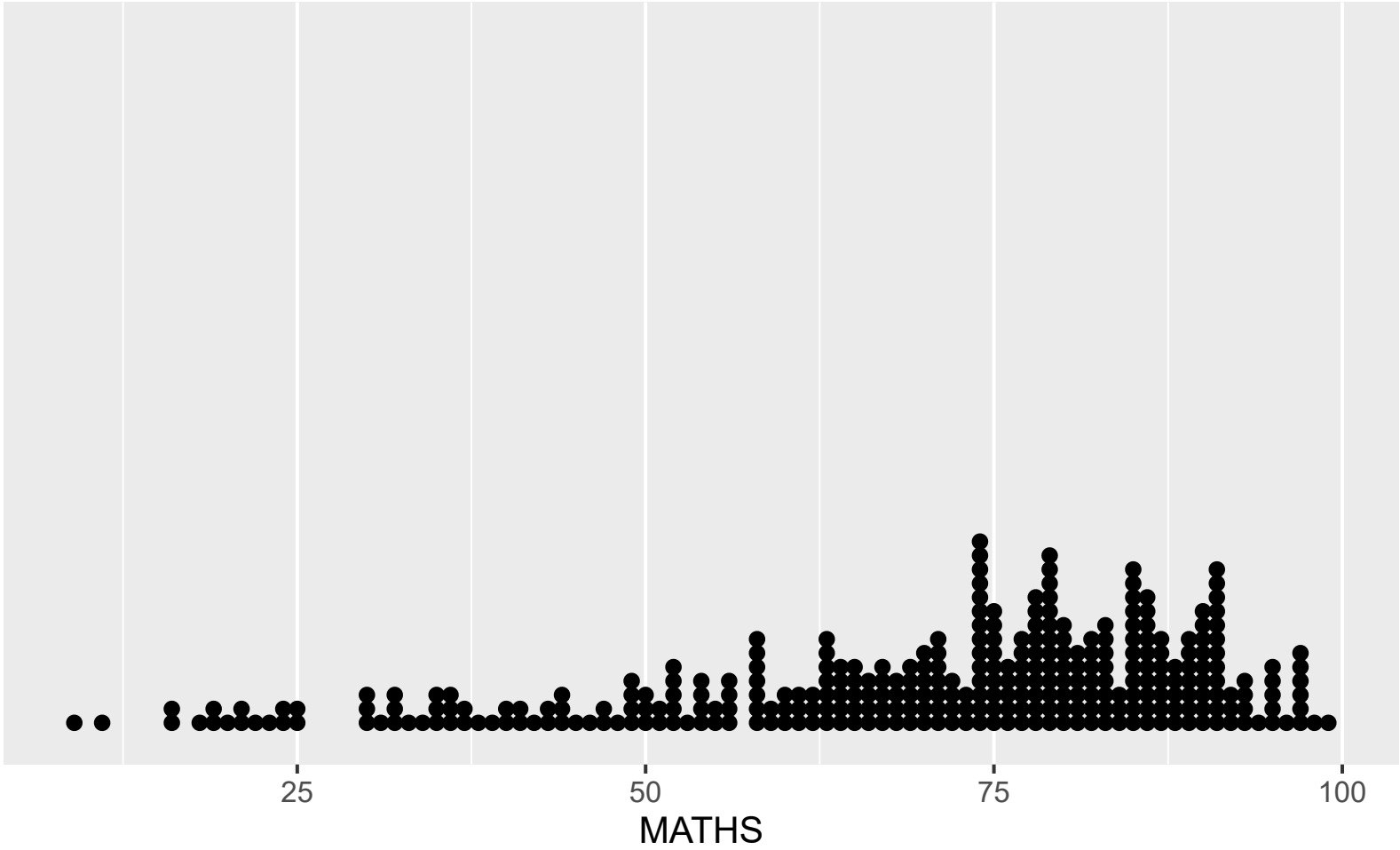
6.5 Combining tooltip and hover effect

There are time that we want to combine tooltip and hover effect on the interactive statistical graph as shown in the code chunk below.

Interactivity: Elements associated with a `data_id` (i.e CLASS) will be highlighted upon mouse over. At the same time, the tooltip will show the CLASS.

```
p <- ggplot(data=exam_data,
  aes(x = MATHS)) +
  geom_dotplot_interactive(
    aes(tooltip = CLASS,
      data_id = CLASS),
    stackgroups = TRUE,
    binwidth = 1,
    method = "histodot") +
  scale_y_continuous(NULL,
    breaks = NULL)

girafe(
  ggobj = p,
  width_svg = 6,
  height_svg = 6*0.618,
  options = list(
    opts_hover(css = "fill: #202020;"),
    opts_hover_inv(css = "opacity:0.2;")
  )
)
```



6.6 Click effect with onclick

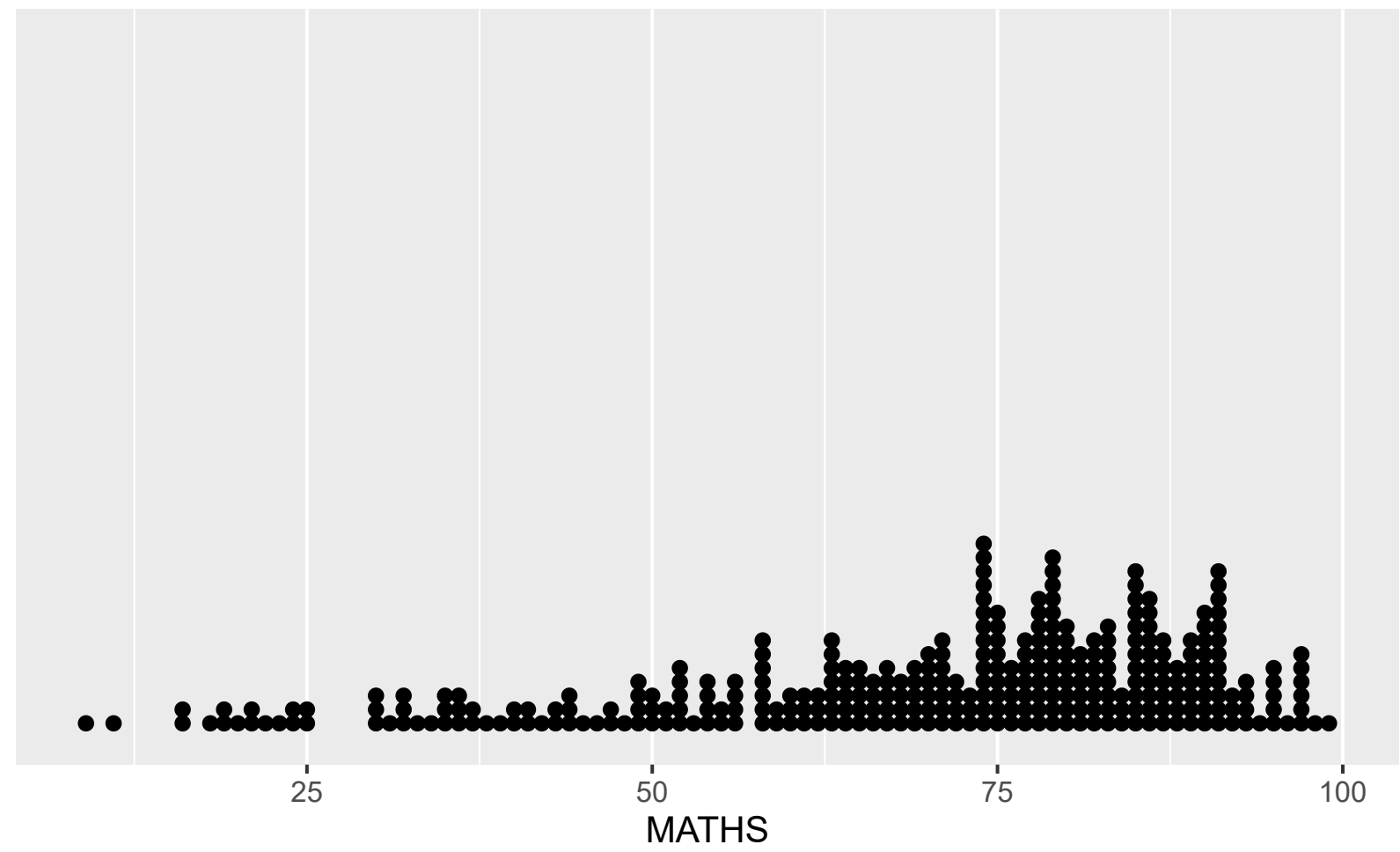
`onclick` argument of `ggiraph` provides hotlink interactivity on the web.

The code chunk below shown an example of `onclick`. Note that click actions must be a string column in the dataset containing valid javascript instructions.

```
exam_data$onclick <- sprintf("window.open(\"%s%s\")",
                             "https://www.moe.gov.sg/schoolfinder?journey=Primary%20school",
                             as.character(exam_data$ID))

p <- ggplot(data=exam_data,
            aes(x = MATHS)) +
  geom_dotplot_interactive(
    aes(onclick = onclick),
    stackgroups = TRUE,
    binwidth = 1,
    method = "histodot") +
  scale_y_continuous(NULL,
                    breaks = NULL)

girafe(
  ggobj = p,
  width_svg = 6,
  height_svg = 6*0.618)
```



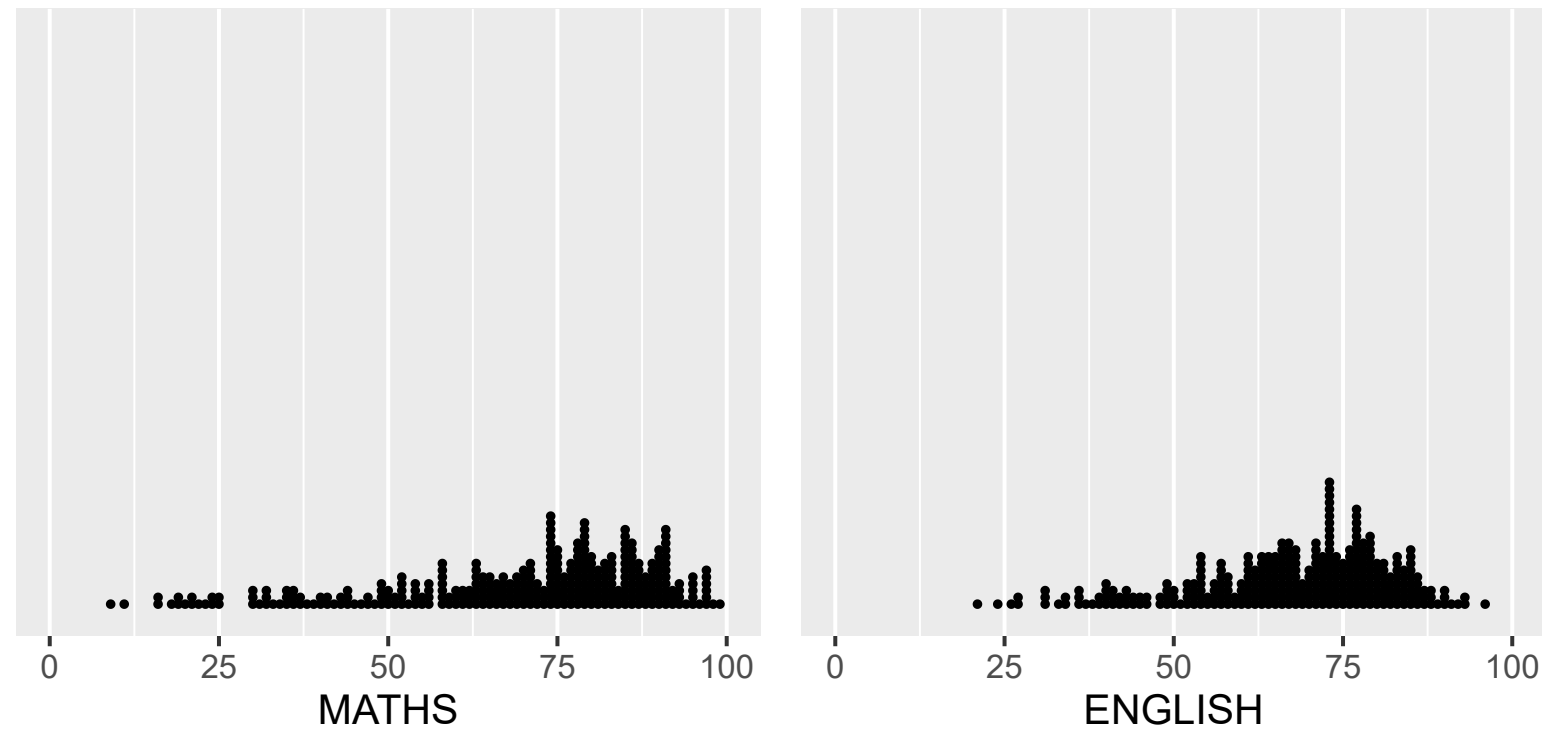
6.7 Coordinated Multiple Views with ggiraph

Coordinated multiple views methods has been implemented in the data visualisation below.

```
p1 <- ggplot(data=exam_data,
             aes(x = MATHS)) +
  geom_dotplot_interactive(
    aes(data_id = ID),
    stackgroups = TRUE,
    binwidth = 1,
    method = "histodot") +
  coord_cartesian(xlim=c(0,100)) +
  scale_y_continuous(NULL,
                    breaks = NULL)

p2 <- ggplot(data=exam_data,
             aes(x = ENGLISH)) +
  geom_dotplot_interactive(
    aes(data_id = ID),
    stackgroups = TRUE,
    binwidth = 1,
    method = "histodot") +
  coord_cartesian(xlim=c(0,100)) +
  scale_y_continuous(NULL,
                    breaks = NULL)

girafe(code = print(p1 + p2),
       width_svg = 6,
       height_svg = 3,
       options = list(
         opts_hover(css = "fill: #202020;"),
         opts_hover_inv(css = "opacity:0.2;")
       )
)
```



Notice that when a data point of one of the dotplot is selected, the corresponding data point ID on the second data visualisation will be highlighted too.

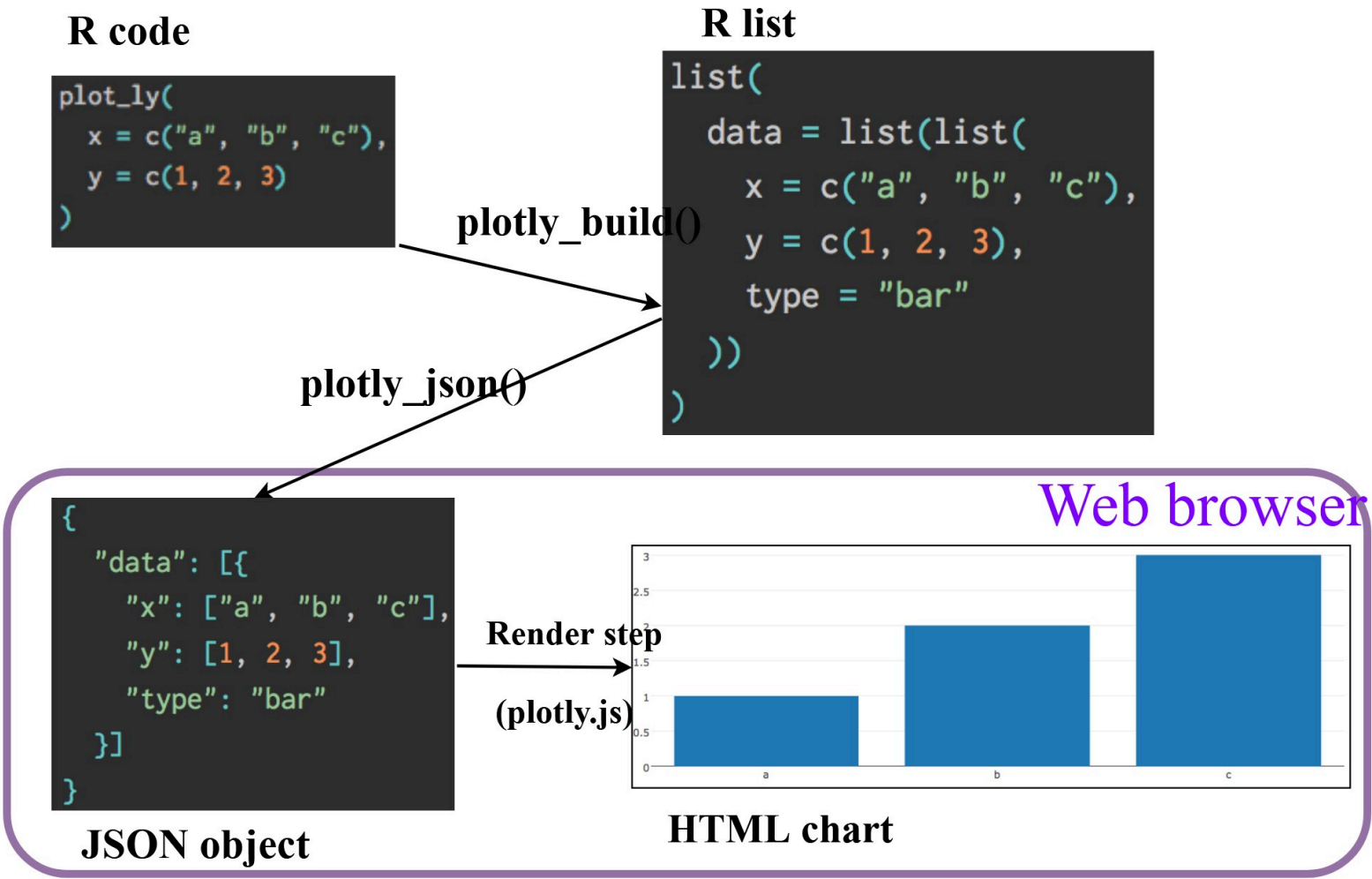
In order to build a coordinated multiple views as shown in the example above, the following programming strategy will be used:

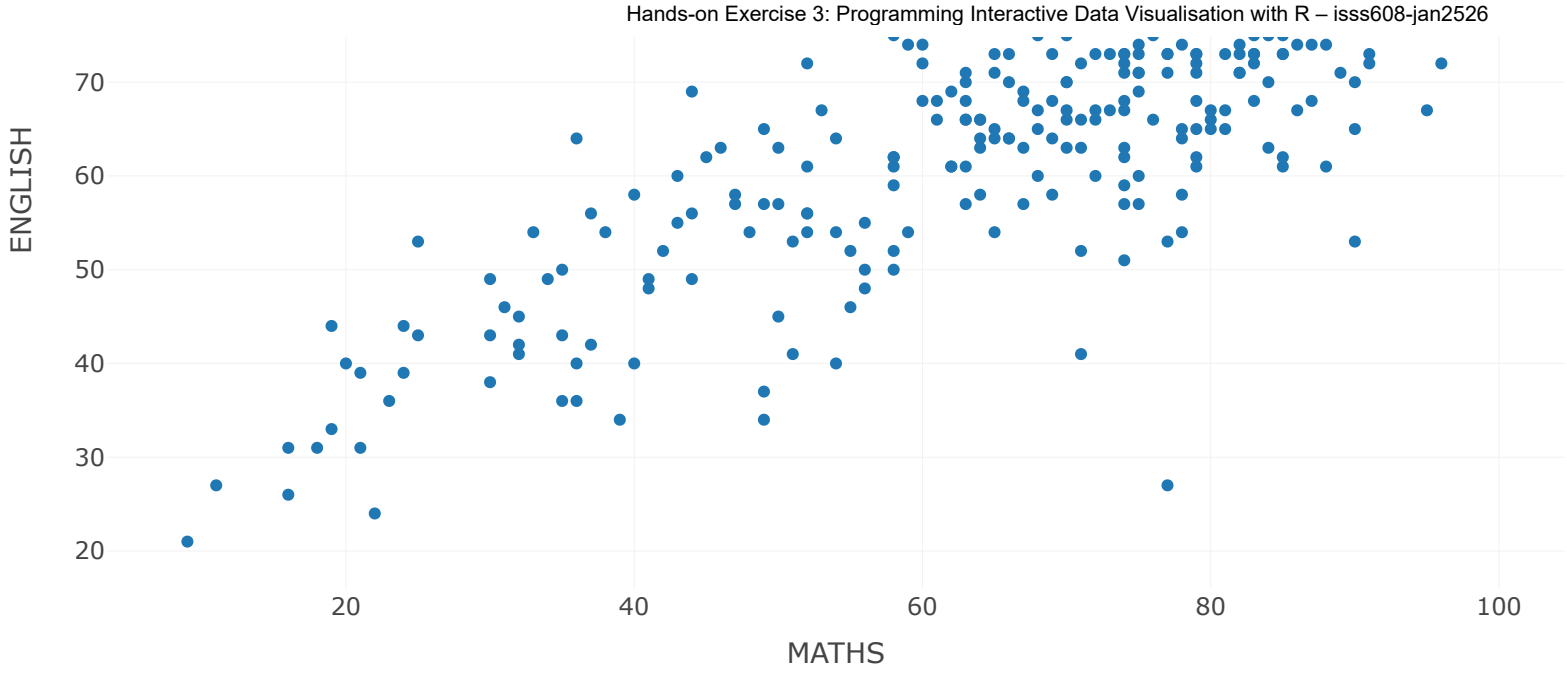
1. Appropriate interactive functions of **ggiraph** will be used to create the multiple views.
2. *patchwork* function of [patchwork](#) package will be used inside girafe function to create the interactive coordinated multiple views.

The *data_id* aesthetic is critical to link observations between plots and the tooltip aesthetic is optional but nice to have when mouse over a point.

7 Interactive Data Visualisation - plotly methods

Plotly's R graphing library create interactive web graphics from **ggplot2** graphs and/or a custom interface to the (MIT-licensed) JavaScript library [plotly.js](#) inspired by the grammar of graphics. Different from other plotly platform, plot.R is free and open source.

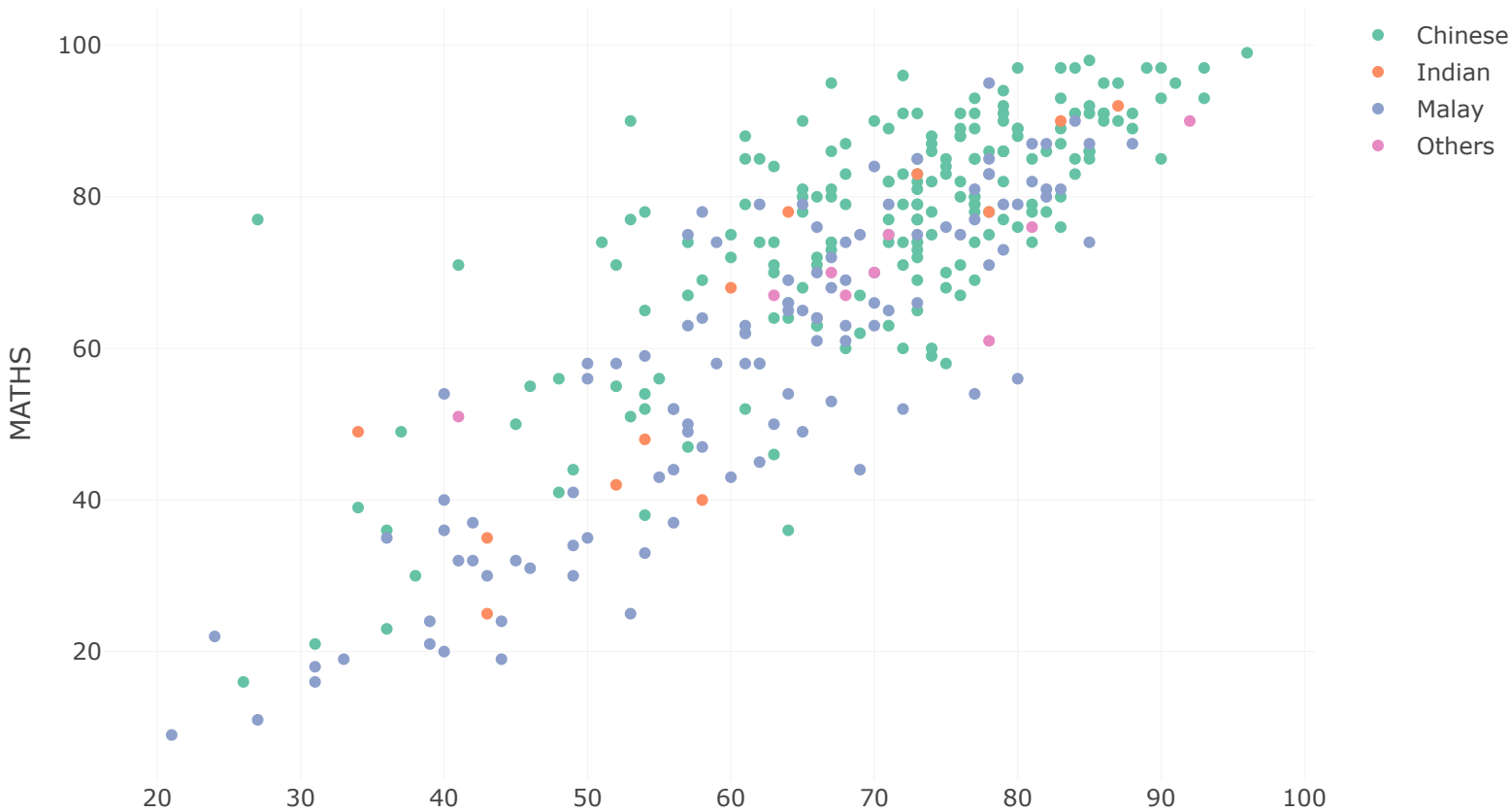




7.2 Working with visual variable: `plot_ly()` method

In the code chunk below, *color* argument is mapped to a qualitative visual variable (i.e. RACE).

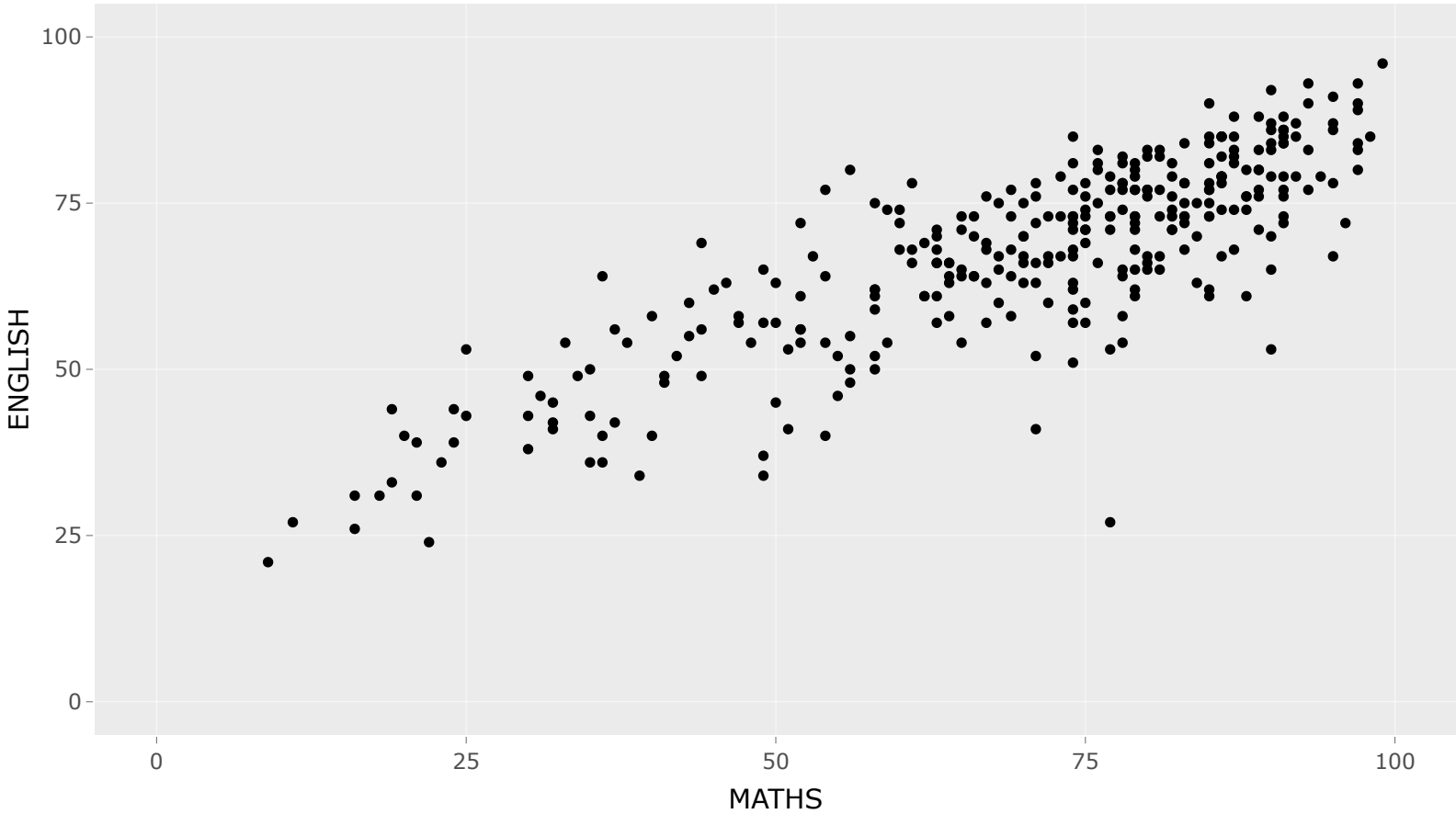
```
plot_ly(data = exam_data,  
  x = ~ENGLISH,  
  y = ~MATHS,  
  color = ~RACE)
```



7.3 Creating an interactive scatter plot: ggplotly() method

The code chunk below plots an interactive scatter plot by using *ggplotly()*. Notice that the only extra line you need to include in the code chunk is *ggplotly()*.

```
p <- ggplot(data=exam_data,
            aes(x = MATHS,
                y = ENGLISH)) +
  geom_point(size=1) +
  coord_cartesian(xlim=c(0,100),
                 ylim=c(0,100))
ggplotly(p)
```



7.4 Coordinated Multiple Views with plotly

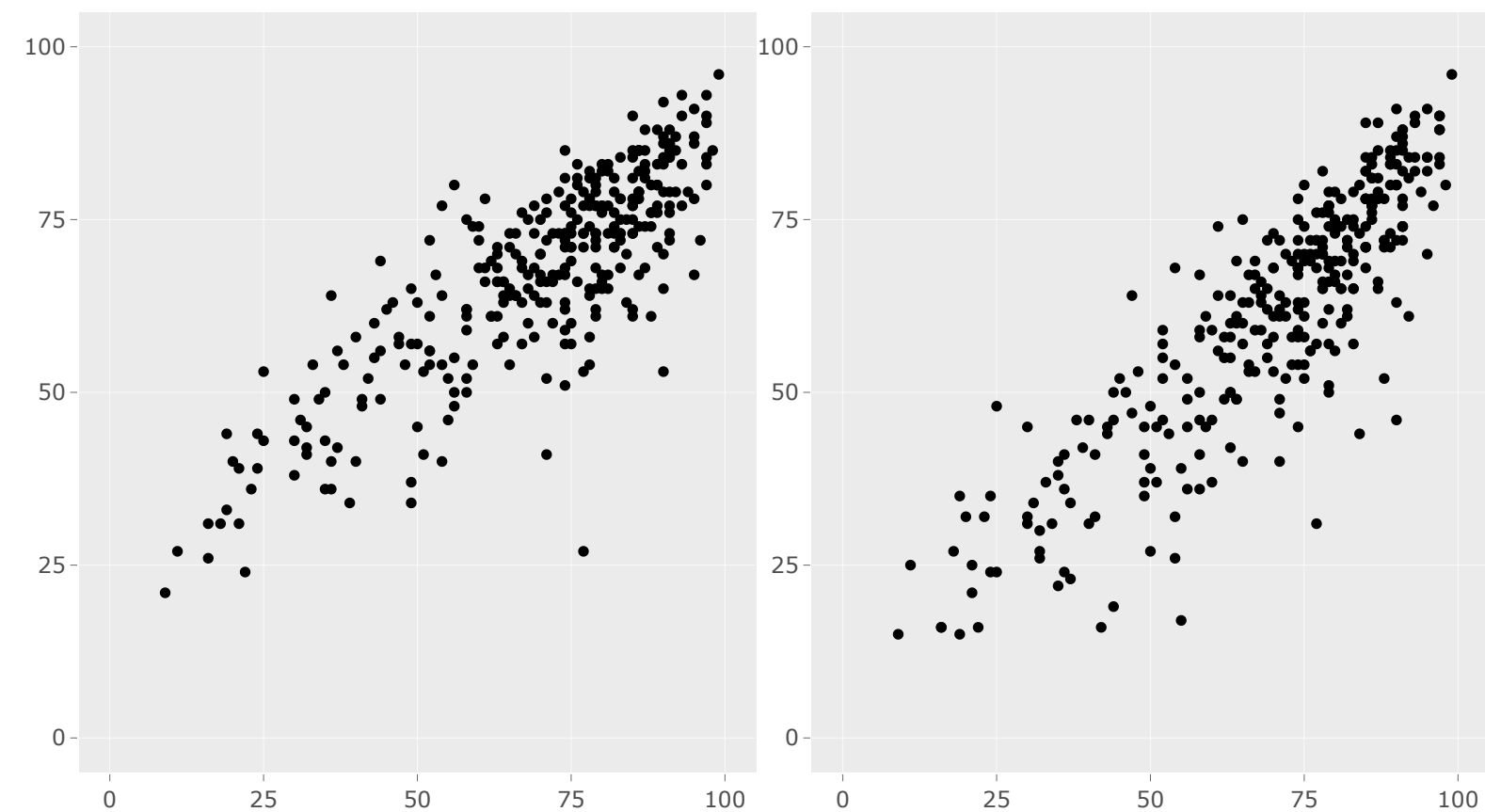
The creation of a coordinated linked plot by using *plotly* involves three steps:

- [highlight_key\(\)](#) of **plotly** package is used as shared data.
- two scatterplots will be created by using *ggplot2* functions.

- lastly, [subplot\(\)](#) of **plotly** package is used to place them next to each other side-by-side.

```
d <- highlight_key(exam_data)
p1 <- ggplot(data=d,
             aes(x = MATHS,
                 y = ENGLISH)) +
  geom_point(size=1) +
  coord_cartesian(xlim=c(0,100),
                 ylim=c(0,100))

p2 <- ggplot(data=d,
             aes(x = MATHS,
                 y = SCIENCE)) +
  geom_point(size=1) +
  coord_cartesian(xlim=c(0,100),
                 ylim=c(0,100))
subplot(ggplotly(p1),
       ggplotly(p2))
```



(*Click on a data point of one of the scatterplot and see how the corresponding point on the other scatterplot is selected.)

Thing to learn from the code chunk:

- `highlight_key()` simply creates an object of class [crosstalk::SharedData](#).
- Visit this [link](#) to learn more about crosstalk

8 Interactive Data Visualisation - crosstalk methods

[Crosstalk](#) is an add-on to the `htmlwidgets` package. It extends `htmlwidgets` with a set of classes, functions, and conventions for implementing cross-widget interactions (currently, linked brushing and filtering).

8.1 Interactive Data Table: DT package

- A wrapper of the JavaScript Library [DataTables](#)
- Data objects in R can be rendered as HTML tables using the JavaScript library ‘DataTables’ (typically via R Markdown or Shiny).

```
DT::datatable(exam_data, class= "compact")
```

Show

10

 entries

Search:

	ID	CLASS	GENDER	RACE	ENGLISH	MATHS	SCIENCE	tooltip	onclick
1	Student321	3I	Male	Malay	21	9	15	Name = Student321 Class = 3I	window.open("https://www.journey=Primary%20school")
2	Student305	3I	Female	Malay	24	22	16	Name = Student305 Class = 3I	window.open("https://www.journey=Primary%20school")
3	Student289	3H	Male	Chinese	26	16	16	Name = Student289 Class = 3H	window.open("https://www.journey=Primary%20school")
4	Student227	3F	Male	Chinese	27	77	31	Name = Student227 Class = 3F	window.open("https://www.journey=Primary%20school")
5	Student318	3I	Male	Malay	27	11	25	Name = Student318 Class = 3I	window.open("https://www.journey=Primary%20school")
6	Student306	3I	Female	Malay	31	16	16	Name = Student306 Class = 3I	window.open("https://www.journey=Primary%20school")
7	Student313	3I	Male	Chinese	31	21	25	Name = Student313 Class = 3I	window.open("https://www.journey=Primary%20school")
8	Student316	3I	Male	Malay	31	18	27	Name = Student316 Class = 3I	window.open("https://www.journey=Primary%20school")

ID	CLASS	GENDER	RACE	ENGLISH	MATHS	SCIENCE	tooltip	onclick				
9	Student312 3I	Male	Malay	33	19	15	Name = Student312 Class = 3I	window.open("https://www.journey=Primary%20school")				
10	Student297 3H	Male	Indian	34	49	37	Name = Student297 Class = 3H	window.open("https://www.journey=Primary%20school")				
Showing 1 to 10 of 322 entries				Previous	1	2	3	4	5	...	33	Next

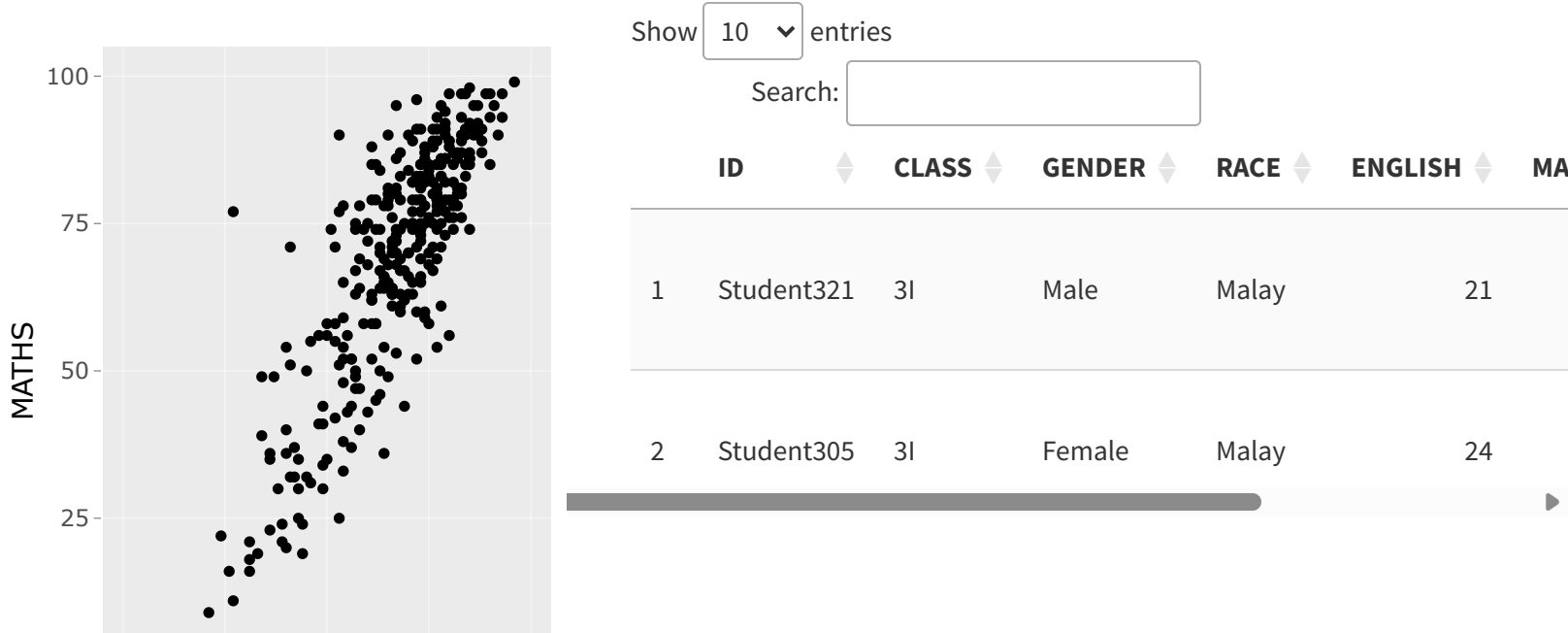
8.2 Linked brushing: crosstalk method

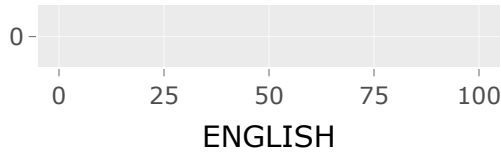
Code chunk below is used to implement the coordinated brushing shown above.

```
d <- highlight_key(exam_data)
p <- ggplot(d,
  aes(ENGLISH,
      MATHS)) +
  geom_point(size=1) +
  coord_cartesian(xlim=c(0,100),
    ylim=c(0,100))

gg <- highlight(ggplotly(p),
  "plotly_selected")

crosstalk::bscols(gg,
  DT::datatable(d),
  widths = 5)
```





	ID		CLASS		GENDER		RACE		ENGLISH		MA
3	Student289		3H		Male		Chinese		26		
4	Student227		3F		Male		Chinese		27		
5	Student318		3I		Male		Malay		27		
6	Student306		3I		Female		Malay		31		
7	Student313		3I		Male		Chinese		31		
8	Student316		3I		Male		Malay		31		
9	Student312		3I		Male		Malay		33		
10	Student297		3H		Male		Indian		34		

Showing 1 to 10 of 322 entries

Things to learn from the code chunk:

- highlight()* is a function of **plotly** package. It sets a variety of options for brushing (i.e., highlighting) multiple plots. These options are primarily designed for linking multiple plotly graphs, and may not behave as expected when linking plotly to another htmlwidget package via crosstalk. In some cases, other htmlwidgets will respect these options, such as persistent selection in leaflet.

- `bscols()` is a helper function of **crosstalk** package. It makes it easy to put HTML elements side by side. It can be called directly from the console but is especially designed to work in an R Markdown document. **Warning:** This will bring in all of Bootstrap!

9 Reference

- Kam, T. S. (2024). *Programming Interactive Data Visualisation with R*. <https://r4va.netlify.app/chap03>

ggiraph

This [link](#) provides online version of the reference guide and several useful articles. Use this [link](#) to download the pdf version of the reference guide.

- [How to Plot With Ggiraph](#)
- [Interactive map of France with ggiraph](#)
- [Custom interactive sunbursts with ggplot in R](#)
- This [link](#) provides code example on how ggiraph is used to interactive graphs for [Swiss Olympians - the solo specialists](#).

plotly for R

- [Getting Started with Plotly in R](#)
- A collection of plotly R graphs are available via this [link](#).
- Carson Sievert (2020) **Interactive web-based data visualization with R, plotly, and shiny**, Chapman and Hall/CRC is the best resource to learn plotly for R. The online version is available via this [link](#)
- [Plotly R Figure Reference](#) provides a comprehensive discussion of each visual representations.
- [Plotly R Library Fundamentals](#) is a good place to learn the fundamental features of Plotly's R API.
- [Getting Started](#)
- Visit this [link](#) for a very interesting implementation of gganimate by your senior.
- [Building an animation step-by-step with gganimate](#).
- [Creating a composite gif with multiple gganimate panels](#)